

Esta unidad dispone de los siguientes archivos:

- Estos mismo materiales en PDF: [04-Analisis_textual-spa.pdf](#)
- Los programas y archivos necesarios: [programas4-spa.zip](#)
- El archivo Notebook de Jupyter: [cap4-spa.ipynb](#)
- El enlace al notebook en Google Colaboratory: <https://colab.research.google.com/github/aoliverg/python/blob/master/notebooks/cap4-spa.ipynb>

4.1. Acceso a los corpus de NLTK

NLTK se distribuye con una gran cantidad de corpus. A pesar de que una gran parte de estos corpus están en inglés, hay también en otras lenguas, como el castellano y el catalán. La lista completa y actualizada de los corpus disponibles en NLTK se puede encontrar en el siguiente enlace: http://www.nltk.org/nltk_data/

Por ejemplo, NLTK incluye una selección de textos del Proyecto Gutenberg. Este proyecto recopila libros en formato electrónico (libros en dominio público, es decir, sin derechos de autor y por tanto legalmente descargables). Si queremos cargar este corpus y observar los archivos que contiene, podemos hacer en un intérprete interactivo:

```
>>> from nltk.corpus import gutenberg
>>> libros=gutenberg.fileids()
>>> print(libros)

['austen-emma.txt', 'austen-persuasion.txt', 'austen-sin.txt', 'bible-kjv.txt', 'blake-poems.txt', 'bryant-stories.txt', 'burgess-busterbrown.txt', 'carroll-alice.txt', 'chesterton-baile.txt', 'chesterton-brown.txt', 'chesterton-thursday.txt', 'edgeworth-parientes.txt', 'melville-moby_dick.txt', 'milton-paradise.txt', 'shakespeare-caesar.txt', 'shakespeare-hamlet.txt', 'shakespeare-macbeth.txt', 'whitman-leaves.txt']
```

Podemos acceder a las palabras de un determinado libro de la colección haciendo lo siguiente:

```
>>> import nltk
>>> mobi=nltk.corpus.gutenberg.words("melville-moby_dick.txt")
>>> print(mobi)

['', 'Moby', 'Dick', 'by', 'Herman', 'Melville', ...]
>>> print(len(mobi))

260819
```

Con el método `words` hemos podido acceder a las palabras del libro (que hemos almacenado a la variable `mobi`). En la última línea hemos contado las posiciones de la lista `mobi`, es decir, el número de palabras del libro. En un corpus de texto como este podemos acceder a:

- al texto en bruto, con el método `raw`
- los párrafos, con el método `paras`
- a las oraciones, con el método `sents`
- a las palabras, con el método `words`

En el programa siguiente (`programa-4-1.py`), podemos observar el funcionamiento de estos métodos (cuando lo ejecutes, para pasar de un nivel al siguiente, pulsa la tecla `Enter`):

```
import nltk

print("TEXTO EN BRUTO")
```

```
a=input()

texto_en_bruto=nltk.corpus.gutenberg.raw("melville-moby_dick.txt")
print(texto_en_bruto[0:100000])

print("PÁRRAFOS")
a=input()

parrafos=nltk.corpus.gutenberg.paras("melville-moby_dick.txt")
for para in parrafos:
    print(para)

print("ORACIONES")
a=input()

oraciones=nltk.corpus.gutenberg.sents("melville-moby_dick.txt")
for oracion in oraciones:
    print(oracion)

print("PALABRAS")
a=input()

palabras=nltk.corpus.gutenberg.words("melville-moby_dick.txt")
for palabra in palabras:
    print(palabra)
```

Con `a=input()` el sistema espera que el usuario introduzca algún valor y lo almacena en `a` (variable que no utilizamos para nada más). Lo que conseguimos es que el sistema se espere hasta que el usuario pulse la tecla Enter.

Por ahora estamos trabajando con un corpus textual plano, sin ningún tipo de análisis ni anotación. Más adelante veremos que NLTK también proporciona corpus con varios niveles de análisis.

4.2. Acceso a corpus propios

NLTK proporciona una buena colección de corpus, pero con toda seguridad necesitaremos también trabajar con corpus propios. Para hacer esto podemos utilizar las instrucciones que ya conocemos para abrir y leer archivos de texto (mira el apartado 2.3 de la Unidad 2), o bien podremos usar los diferentes lectores de corpus que proporciona NLTK.

El lector de corpus más elemental que proporciona el NLTK es el **PlaintextCorpusReader**, que sirve, como indica su nombre, para leer corpus que estén en formato de texto plano (es decir, solo el texto, sin ningún tipo de anotación). En el programa-4-2.py se puede observar el funcionamiento básico. En este caso también usaremos la novela Moby Dick, descargada del Proyecto Gutenberg y que podemos encontrar en los ficheros de este capítulo.

```
from nltk.corpus.reader.plaintext import PlaintextCorpusReader

corpus = PlaintextCorpusReader(".", 'mobi_dick.txt')
```

```
for oracion in corpus.sents():  
    print(oracion)
```

y en la salida tendremos las oraciones, que de hecho, si te fijas bien, son listas de palabras:

```
...  
['In', 'a', 'word', ',', 'the', 'whale', 'was', 'seized', 'and', 'sold', ',', 'and', 'his', 'Grace', 'the', 'Duke', 'of', 'Wellington',  
'received', 'the', 'money', '.']  
...
```

El **PlaintextCorpusReader** carga el fichero de texto y lleva a cabo un proceso de segmentación en párrafos (a partir de saltos de párrafos), en oraciones (usando un segmentador determinado) y en palabras (o tokens, usando un tokenizador determinado). Si no indicamos nada en el momento de llamar a `PlaintextCorpusReader` utiliza:

- Tokenizador: `word_tokenizer=WordPunctTokenizer()`,
- Segmentador: `sent_tokenizer=nlk.data.LazyLoader('tokenizers/punkt/english.pickle')` Es decir, asume por defecto que la lengua del corpus es el inglés.

Como que el tokenizador que usa es muy simple y se basa en separar el texto en secuencias de caracteres alfabéticos y no alfabéticos usando la expresión regular `\w+|[\^\w\s]+` se pueden producir errores, como el siguiente:

```
['"', 'Won', "'", 't', 'the', 'Duke', 'be', 'content', 'with', 'a', 'quarter', 'or', 'a', 'half', '?"']
```

dónde `won't` se ha separado como

```
won  
,  
t
```

Fijémonos ahora que si aplicamos este mismo programa a un corpus en castellano la tokenización no será totalmente correcta. Con este capítulo se distribuye un fragmento del Corpus del Diario Oficial de la Generalitat de Cataluña (corpus DOGC), concretamente el correspondiente a la versión castellana del año 2015. Si cargamos el fichero `DOGC-2015-spa.txt`, en vez de `mobi_dick.txt` obtenemos un resultado como el siguiente (mostramos únicamente un fragmento):

```
...  
['El', 'pleno', 'del', 'Ayuntamiento', 'de', 'Òrrius', 'en', 'la', 'sesión', 'extraordinaria', 'celebrada', 'el', 'día', '20', 'de',  
'enero', 'de', '2015', 'aprobó', 'inicialmente', 'el', 'proyecto', 'de', 'obra', ...]  
...
```

Para el castellano el segmentador funciona bastante bien. Si probamos ahora con el mismo texto en catalán (`DOGC-2015-cat.txt`), veremos que se producen algunos problemas:

```
...  
['El', 'ple', 'de', 'l', 'Ajuntament', 'd', 'Òrrius', 'a', 'la', 'sessió', 'extraordinària', 'celebrada', 'el', 'dia', '20', 'de',  
'gener', 'de', '2015', 'va', 'aprovar', 'inicialment', 'el', 'projecte', 'd', 'obra', ...]  
...
```

Fijémonos que, por ejemplo, los apóstrofes (') quedan como tokens aislados, separados del artículo o preposición correspondiente. En este mismo capítulo, en los apartados 4.4 y 4.5, tratamos más a fondo la segmentación en unidades léxicas (tokenización) y la segmentación en oraciones. Lo que sí que avanzamos ahora es que al `PlaintextCorpusReader` se le puede indicar qué tokenizador y segmentador tiene que utilizar. Esto

se puede hacer de la siguiente manera (programa-4-3.py)(necesitarás descargar el archivo catalan.pickle que tenéis en la sección de archivos de este capítulo):

```
import nltk

from nltk.corpus.reader.plaintext import PlaintextCorpusReader

from nltk.tokenize import RegexpTokenizer

segmentador= nltk.data.load("catalan.pickle")

tokenizador=RegexpTokenizer('[ldsmLDSM]'\w+[^\w\s]+')

corpus = PlaintextCorpusReader(".", 'DOGC-2015-
cat.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)

for oracion in corpus.sents():

    print(oracion)
```

Fíjate que hemos definido un segmentador (que cargamos de catalan.pickle) y un tokenizador basado en expresiones regulares, y que usamos estos nuevos elementos cuando creamos el PlaintextCorpusReader. Ahora hemos arreglado el aspecto de los apóstrofes en la tokenización y la salida (mostramos solo un fragmento) es:

```
['El', 'ple', 'de', 'l"', 'Ajuntament', 'd"', 'Òrrius', 'a', 'la', 'sessió', 'extraordinària', 'celebrada', 'el', 'dia', '20', 'de',
'gener', 'de', '2015', 'va', 'aprovar', 'inicialment', 'el', 'projecte', 'd"', 'obra', '.']
```

En el apartado 4.4 veremos con más detalle el tema de las expresiones regulares. Comparamos ahora las expresiones regulares del tokenizador por defecto:

```
\w+[^\w\s]+
```

y el que hemos creado nosotros:

```
[ldsmLDSM]'\w+[^\w\s]+
```

Hemos añadido la parte [ldsmLDSM]' que define los tokens formados por: l', d', s', m', L', D', S', M'.

4.3. Ocurrencias (tokens) y tipos (types)

Modificaremos ligeramente el programa-4-3.py, para que nos dé todas las palabras (o tokens) en vez de todas las oraciones y para que nos proporcione el número total de palabras (programa-4-4.py):

```
import nltk

from nltk.corpus.reader.plaintext import PlaintextCorpusReader

from nltk.tokenize import RegexpTokenizer

segmentador= nltk.data.load("catalan.pickle")

tokenizador=RegexpTokenizer('[ldsmLDSM]'\w+[^\w\s]+')

corpus = PlaintextCorpusReader(".", 'DOGC-2015-
cat.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)

for paraula in corpus.words():
```

```
print(paraula)
print("TOTAL PALABRAS:",len(corpus.words()))
```

El programa se está un buen rato mostrando palabras por pantalla y después nos da el número de palabras totales:

```
...
5524
,
de
11
.
12
.
2009
),
TOTAL PALABRAS: 2448870
```

Si miramos con detenimiento la lista veremos que muchas de las palabras ocurren más de una vez (en el apartado 4.6 veremos como calcular la frecuencia y la distribución de frecuencias de las palabras). Si ahora lo que queremos es obtener una lista de palabras diferentes, podemos utilizar la instrucción set, como podemos observar en el programa-4-5.py:

```
import nltk

from nltk.corpus.reader.plaintext import PlaintextCorpusReader
from nltk.tokenize import RegexpTokenizer

segmentador= nltk.data.load("catalan.pickle")
tokenitzador=RegexpTokenizer('[\dsmLDSM]'\w+|^[^\w\s]+')

corpus = PlaintextCorpusReader(".", 'DOGC-2015-
cat.txt',word_tokenizer=tokenitzador,sent_tokenizer=segmentador)

ocurrencies=corpus.words()
tipus=set(ocurrencies)

print("OCURRENCIAS:",len(ocurrencies))
print("TIPOS:",len(tipus))
```

Que nos da en la salida:

```
OCURRENCIAS: 2448870
TIPOS: 34872
```

Para resumir podemos decir que las ocurrencias son el número total de palabras que aparecen en el corpus y los tipos el número de palabras diferentes que aparecen en el corpus. Hay que tener en cuenta, no obstante, que no podemos hablar estrictamente de palabras, puesto que también se incluyen los signos de puntuación, las cifras, etc. Podemos calcular un índice de riqueza léxica dividiendo el número de ocurrencias entre el número de tipos. Lo vemos al programa-4-6.py:

```
import nltk

from nltk.corpus.reader.plaintext import PlaintextCorpusReader
from nltk.tokenize import RegexpTokenizer

segmentador= nltk.data.load("catalan.pickle")
tokenizador=RegexpTokenizer('[\dsmLDSM]'\w+|['^'\w\s]+'')

corpus = PlaintextCorpusReader(".", 'DOGC-2015-
cat.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)

ocurrencias=corpus.words()
tipos=set(ocurrencias)
riquezalexica=len(ocurrencias)/len(tipos)

print("OCURRENCIAS:",len(ocurrencias))
print("TIPOS:",len(tipos))
print("RIQUEZA LÉXICA:",round(riquezalexica,2))
```

Que nos da a la salida:

OCURRENCIAS: 2448870

TIPOS: 34872

RIQUEZA LÉXICA: 70.22

Lo que indica que cada palabra de media se usa 70 veces. Experimenta un poco con diferentes tipos de texto para ver cómo varía este índice de riqueza léxica.

4.4. Tokenización

La segmentación en unidades léxicas o tokenización, consiste en dividir el texto en unidades más pequeñas (que a menudo coinciden con palabras). A pesar de que se trata de una tarea muy básica y necesaria para poder llevar a cabo tareas de análisis más avanzadas, esta tarea presenta numerosos problemas que no son fáciles de solucionar. Hay numerosos trabajos sobre esta área entre los cuales se pueden destacar los trabajos de [Grefenstette y Tapanainen \(1994\)](#) y [Mikheev \(2002\)](#). Sea como sea, actualmente se puede considerar que esta tarea se resuelve de manera satisfactoria y no hay una investigación activa para mejorarla. En este apartado aprenderemos algunas técnicas para segmentar el texto en unidades léxicas e iremos observando los diferentes problemas que aparecen y cómo se pueden solucionar. Las pruebas de los diferentes sistemas los haremos con la oración:

El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación.

A pesar de que el resultado deseado de la tokenización puede depender de las tareas concretas, la salida deseada de nuestro sistema tendría que ser algo del siguiente estilo:

```
['El', 'Sr.', 'Martínez', 'llegará', 'mañana', 'de', 'Alicante', 'con', 'la', 'R.E.N.F.E.', 'a', 'las', '22.30', 'h.', 'y', 'se', 'alojará', 'en', 'el', 'hotel', 'de', 'la', 'estación', '.']
```

El primer tokenizador que probaremos utiliza la instrucción `split()`, que divide una cadena según el separador que se indique, y si no se indica nada, por espacios. Lo podemos ver al programa-4-7.py:

```
oracion="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación."
```

```
tokens=oracion.split()
```

```
print(tokens)
```

que nos da la siguiente salida, que no es exactamente la que queríamos:

```
['El', 'Sr.', 'Martínez', 'llegará', 'mañana', 'de', 'Alicante', 'con', 'la', 'R.E.N.F.E.', 'a', 'las', '22.30', 'h.', 'y', 'se', 'alojará', 'en', 'el', 'hotel', 'de', 'la', 'estación.']
```

NLTK proporciona una serie de tokenizadores que presentamos a continuación.

WhitespaceTokenizer

Separa el texto por espacios en blanco, como hemos hecho en el ejemplo anterior. En este caso los espacios en blanco pueden ser los caracteres: espacio en blanco, tabulador y nueva línea. Lo podemos ver en el programa-4-8.py:

```
import nltk
```

```
oracion="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación."
```

```
tokens=nltk.tokenize.WhitespaceTokenizer().tokenize(oracion)
```

```
print(tokens)
```

Que da como salida:

```
['El', 'Sr.', 'Martínez', 'llegará', 'mañana', 'de', 'Alicante', 'con', 'la', 'R.E.N.F.E.', 'a', 'las', '22.30', 'h.', 'y', 'se', 'alojará', 'en', 'el', 'hotel', 'de', 'la', 'estación.']
```

Aprovecho este ejemplo para explicar varias maneras de importar un tokenizador, o cualquier método de una clase determinada. En el caso anterior hemos importado todo el NLTK y hemos llamado al método haciendo:

```
tokens=nltk.tokenize.WhitespaceTokenizer().tokenize(oracion)
```

Esto también se puede hacer de la siguiente manera alternativa (programa-4-8b.py) (fijaos como accedemos ahora al método `tokenize`)

```
from nltk.tokenize import WhitespaceTokenizer
```

```
oracion="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación."
```

```
tokens=WhitespaceTokenizer().tokenize(oracion)
```

```
print(tokens)
```

y también de la siguiente manera, dando un nombre cualquiera al método (programa-4-8c.py)

```
from nltk.tokenize import WhitespaceTokenizer as tokenizador

oracion="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación."

tokens=tokenizador().tokenize(oracion)

print(tokens)
```

SpaceTokenizer

Es igual que el anterior, pero en este caso el único carácter que se tiene en cuenta es el de espacio en blanco (" "). Equivale a **split(" ")**. En nuestro ejemplo la salida sería exactamente la misma por lo que no es necesario proporcionar ni el código ni el programa.

TreebankWordTokenizer

Este tokenizador utiliza las convenciones del Penn Treebank corpus, qué es un corpus anotado del inglés creado en 1980 a partir de artículos del Wall Street Journal. Cómo que es para el inglés, no funcionará del todo bien para otras lengua y por este motivo en el ejemplo pongo una oración del inglés (programa-4-9.py):

```
from nltk.tokenize import TreebankWordTokenizer as tokenizador

oracion="We need to conduct an assessment to learn whether a student's difficulties are because he or she can't or won't complete assignments."

tokens=tokenizador().tokenize(oracion)

print(tokens)
```

y la salida será:

```
['We', 'need', 'to', 'conduct', 'an', 'assessment', 'to', 'learn', 'whether', 'a', 'student', "'s", 'difficulties', 'are', 'because', 'he', 'or', 'she', 'ca', "'n't", 'or', 'wo', "'t", 'complete', 'assignments', '.']
```

Este tokenizador se usa bastante para el inglés, y por este motivo se ha creado una función específica que hace de wrapper (wrapper function) para simplificar su uso (programa-4-9b.py):

```
from nltk.tokenize import word_tokenize

oracion="We need to conduct an assessment to learn whether a student's difficulties are because he or she can't or won't complete assignments."

tokens=word_tokenize(oracion)

print(tokens)
```

que proporciona exactamente la misma salida.

RegexpTokenizer

En el apartado anterior (4.3. Ocurrencias (tokens) y tipos (types)) ya vimos un ejemplo del tokenizador por expresiones regulares. Este es un tipo de tokenizador que nos permite un control total sobre el proceso de tokenización. Claro que para sacar el máximo provecho hay que dominar las expresiones regulares de Python. Podéis encontrar una explicación detallada en [Regular Expression HOWTO](#) y también un buen resumen a

4. Análisis textual y procesamiento de corpus

Pyschools.com (<http://doc.pyschools.com/html/regex.html>). Otro buen recurso para trabajar con expresiones regulares es la web <http://regexr.com/>.

Veamos ahora algunos ejemplos, empezando por el programa-4-10.py.

```
from nltk.tokenize import RegexpTokenizer

oracion="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación."

tokenizador=RegexpTokenizer('\w+|^\w\s+')
tokens=tokenizador.tokenize(oracion)
print(tokens)
```

Que nos ofrece la siguiente salida:

```
['El', 'Sr', '.', 'Martínez', 'llegará', 'mañana', 'de', 'Alicante', 'con', 'la', 'R', '.', 'E', '.', 'N', '.', 'F', '.', 'E', '.', 'a', 'las', '22', '.', '30', 'h', '.', 'y', 'se', 'alojará', 'en', 'el', 'hotel', 'de', 'la', 'estación', '.']
```

Cómo podemos observar, nos separa Sr del punto (.) y realmente querríamos tener Sr. como token, y lo mismo pasa con h.. También tokeniza incorrectamente R.E.N.F.E. Podríamos solucionar esto modificando la expresión regular (programa-4-10b.py) de forma que añadimos estas dos unidades como tokens.

```
tokenizador=RegexpTokenizer('Sr\.|h\.|R\.|E\.|N\.|F\.|E\.|[ldsmLDSM]'\w+|^\w\s+')
```

que nos devolvería la tokenización correcta de estas dos unidades:

```
['El', 'Sr.', 'Martínez', 'llegará', 'mañana', 'de', 'Alicante', 'con', 'la', 'R.E.N.F.E.', 'a', 'las', '22', '.', '30', 'h.', 'y', 'se', 'alojará', 'en', 'el', 'hotel', 'de', 'la', 'estación', '.']
```

El hecho de añadir una lista de abreviaturas frecuentes es bastante habitual, pero no podemos esperar tener una lista suficiente completa de acrónimos. Por este motivo, tenemos que intentar expresar los acrónimos de una manera más general, cómo por ejemplo la que presentamos al programa-4-10c.py

```
tokenizador=RegexpTokenizer('Sr\.|h\.|[A-Z]{2,}\w+|^\w\s+')
```

que nos ofrece la misma salida pero que ahora contempla cualquier acrónimo tipo R.E.N.F.E.

Por defecto, el tokenizador por expresiones regulares considera que lo que expresamos son los tokens, pero podemos definir el que son los separadores de tokens con el modificador **gaps=True**. En el programa-4-10.py podemos ver un ejemplo simple.

```
import re

from nltk.tokenize import RegexpTokenizer

oracion="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación."

tokenizador=RegexpTokenizer('\s+',gaps=True)
tokens=tokenizador.tokenize(oracion)
print(tokens)
```

Que da la siguiente salida (que coincide con la del tokenizador por espacios en blanco):

['El', 'Sr.', 'Martínez', 'llegará', 'mañana', 'de', 'Alicante', 'con', 'la', 'R.E.N.F.E.', 'a', 'las', '22.30', 'h.', 'y', 'se', 'alojará', 'en', 'el', 'hotel', 'de', 'la', 'estación.']

4.5. Segmentación

En la sección anterior hemos visto como separar el texto en unidad léxicas, proceso que también recibe el nombre de tokenización. En esta sección veremos cómo podemos separar un párrafo en segmentos, que son unidades parecidas a oraciones. Este proceso recibe el nombre de segmentación.

En esta sección usaremos dos segmentos de prueba, uno que no presentará grandes problemas:

Hoy hace un día muy bonito. Mañana Albert irá a comer en casa.

y otro que sí que presenta numerosos problemas.

El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación. El día siguiente visitará al Dr. Rovira a la Avda. Tibidabo. La vuelta la hará en avión en el vuelo AF.352.

PunktSentenceTokenizer

Este es un segmentador sencillo que básicamente segmenta por puntos. Para ver cómo funciona probamos el programa-4-11.py

```
from nltk.tokenize import PunktSentenceTokenizer
```

```
parrafo1="Hoy hace un día muy bonito. Mañana Albert irá a comer en casa."
```

```
parrafo2="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación. El día siguiente visitará al Dr. Rovira en la Avda. Tibidabo. La vuelta la hará en avión en el vuelo AF.352."
```

```
segmentador=PunktSentenceTokenizer()
```

```
segmentos1=segmentador.tokenize(parrafo1)
```

```
print(segmentos1)
```

```
segmentos2=segmentador.tokenize(parrafo2)
```

```
print(segmentos2)
```

Que proporciona la siguiente salida:

```
['Hoy hace un día muy bonito.', 'Mañana Albert irá a comer en casa.']
```

```
['El Sr.', 'Martínez llegará mañana de Alicante con la R.E.N.F.E.', 'a las 22.30 h. y se alojará en el hotel de la estación.', 'El día siguiente visitará al Dr.', 'Rovira a la Avda.', 'Tibidabo.', 'La vuelta la hará en avión en el vuelo AF.352.']
```

sent_tokenize()

El proceso de segmentación también se puede hacer con **sent_tokenizer()**, que llama a una instancia especial del **PunktSenteceTokenizer** que ha sido entrenada y que funciona bastante bien para varias lenguas europeas. Se trata de una implementación del algoritmo de [Kiss and Strunk \(2006\)](#). Lo podemos ver en el programa-4-11b.py.

```
from nltk.tokenize import sent_tokenize
```

```
parrafo1="Hoy hace un día muy bonito. Mañana Albert irá a comer en casa."
```

```
parrafo2="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación. El día siguiente visitará al Dr. Rovira en la Avda. Tibidabo. La vuelta la hará en avión en el vuelo AF.352."
```

```
segmentos1=sent_tokenize(parrafo1)
```

```
print(segmentos1)
```

```
segmentos2=sent_tokenize(parrafo2)
```

```
print(segmentos2)
```

Que nos ofrece una salida mucho mejor, a pesar de que no perfecta:

```
['Hoy hace un día muy bonito.', 'Mañana Albert irá a comer en casa.']
```

```
['El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E.', 'a las 22.30 h. y se alojará en el hotel de la estación.', 'El día siguiente visitará al Dr. Rovira en la Avda.', 'Tibidabo.', 'La vuelta la hará en avión en el vuelo AF.352.']
```

Cargar un segmentador concreto para una lengua determinada

Con los datos del NLTK se distribuyen una serie de segmentadores para lenguas determinadas. Concretamente se distribuyen los siguientes: checo, finlandés, noruego, español, danés, francés, polaco, sueco, holandés, alemán, portugués, turco, inglés, griego, estonio, italiano, esloveno. No se distribuye uno para el catalán, pero en esta misma sección aprenderemos a crear uno específico para el catalán. En el programa-4-12.py cargamos un de específico para el inglés:

```
import nltk.data
```

```
parrafo="Today Mr. Smith and Ms. Johanson will meet at St. Patrick church."
```

```
segmentador=nltk.data.load("tokenizers/punkt/PY3/english.pickle")
```

```
segmentos=segmentador.tokenize(parrafo)
```

```
print(segmentos)
```

Que proporciona la salida:

```
['Today Mr. Smith and Ms. Johanson will meet at St. Patrick church.']
```

Entrenamiento de un segmentador

El algoritmo de [Kiss and Strunk \(2006\)](#) permite entrenar un segmentador a partir de texto sin ningún tipo de anotación. NLTK implementa este algoritmo. Vamos a entrenar un segmentador para el castellano (de hecho crear un `spanish.pickle`, aunque ya se distribuye uno con NLTK) a partir del corpus de DOGC correspondiendo al año 2015. El programa-4-13.py implementa este aprendizaje:

```
import nltk.tokenize.punkt
```

```
import pickle
```

```
import codecs
```

```
segmentador = nltk.tokenize.punkt.PunktSentenceTokenizer()
```

```
texto = codecs.open("DOGC-2015-spa.txt", "r", "utf8").read()
```

```
segmentador.train(texto)
```

```
out = open("spanish.pickle","wb")
pickle.dump(segmentador, out)
out.close()
```

El segmentador almacenado en spanish.pickle lo podemos utilizar programa-4-12b.py):

```
import nltk.data
segmentador=nltk.data.load("spanish.pickle")
parrafo1="Hoy hace un día muy bonito. Mañana Albert irá a comer en casa."
parrafo2="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación. El día siguiente visitará al Dr. Rovira en la Avda. Tibidabo. La vuelta la hará en avión en el vuelo AF.352."
```

```
segmentos1=segmentador.tokenize(parrafo1)
print(segmentos1)
segmentos2=segmentador.tokenize(parrafo2)
print(segmentos2)
```

Que ofrece la siguiente salida:

```
['Hoy hace un día muy bonito.', 'Mañana Albert irá a comer en casa.']
```

```
['El Sr.', 'Martínez llegará mañana de Alicante con la R.E.N.F.E.', 'a las 22.30 h. y se alojará en el hotel de la estación.', 'El día siguiente visitará al Dr.', 'Rovira en la Avda.', 'Tibidabo.', 'La vuelta la hará en avión en el vuelo AF.352.']
```

¿Funciona del todo correctamente el segmentador que hemos entrenado?

Personalizar el segmentador

Podemos personalizar el segmentador entrenado para añadir nuevas abreviaturas o acrónimos que no han sido detectados en el proceso de entrenamiento. Lo podemos hacer específicamente para un programa determinado, cómo en el programa-4-12c.py, donde cargamos el spanish.pickle que hemos entrenado y añadimos R.E.N.F.E. (fíjate que lo hacemos en minúsculas y sin el punto final):

```
import nltk.data
segmentador=nltk.data.load("spanish.pickle")
abreviaturas_extra = ['r.e.n.f.e']
segmentador._params.abbrev_types.update(abreviaturas_extra)
parrafo1="Hoy hace un día muy bonito. Mañana Albert irá a comer en casa."
parrafo2="El Sr. Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación. El día siguiente visitará al Dr. Rovira en la Avda. Tibidabo. La vuelta la hará en avión en el vuelo AF.352."
segmentos1=segmentador.tokenize(parrafo1)
print(segmentos1)
segmentos2=segmentador.tokenize(parrafo2)
print(segmentos2)
```

y ahora la salida ya segmenta mejor:

['El Sr.', 'Martínez llegará mañana de Alicante con la R.E.N.F.E. a las 22.30 h. y se alojará en el hotel de la estación.', 'El día siguiente visitará al Dr.', 'Rovira en la Avda.', 'Tibidabo.', 'La vuelta la hará en avión en el vuelo AF.352.']

En los ficheros adjuntos podéis encontrar una lista de abreviaciones. Ahora lo que haremos será cargar el `spanish.pickle` que hemos entrenado y modificarlo añadiendo la lista de abreviaturas y acrónimos del fichero. Grabaremos este nuevo segmentador como `spanish-mod.pickle`. (`programa-4-14.py`)

```
import nltk.data
import codecs
import pickle
segmentador=nltk.data.load("spanish.pickle")
archivo_abreviaturas=codecs.open("abr-spa.txt","r",encoding="utf-8")
abreviaturas_extra=[]
for abreviatura in archivo_abreviaturas.readlines():
    abreviatura=abreviatura.rstrip()
    abreviaturas_extra.append(abreviatura)
segmentador._params.abbrev_types.update(abreviaturas_extra)
out = open("spanish-mod.pickle","wb")
pickle.dump(segmentador, out)
out.close()
```

Modifica el `programa-4-12b.py` para que cargue este nuevo segmentador y verifica si funciona correctamente.

4.6. Frecuencias y distribución de frecuencias

En este apartado aprenderemos a realizar algunos cálculos sencillos sobre corpus: frecuencias absolutas y frecuencias relativas, distribuciones de frecuencia y a encontrar las colocaciones más frecuentes de un corpus.

4.6.1. Frecuencia absoluta

Entendemos por frecuencia absoluta el número total de veces que aparece una determinada unidad léxica en nuestro corpus.

El cálculo de la frecuencia absoluta de una palabra es sencillo: podemos utilizar un diccionario para poner como clave las palabras e ir incrementando el valor del diccionario cada vez que aparece la palabra. En el siguiente programa (`programa-4-15.py`) podemos ver una implementación sencilla de esta idea:

```
import nltk
from nltk.corpus.reader.plaintext import PlaintextCorpusReader
from nltk.tokenize import RegexpTokenizer
segmentador= nltk.data.load("spanish-mod.pickle")
tokenizador=RegexpTokenizer("\w+|^[^\w\s]+")
```

```
corpus = PlaintextCorpusReader(".", 'DOGC-2015-  
spa.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)  
  
frecuencia={}  
  
for palabra in corpus.words():  
    frecuencia[palabra]=frecuencia.get(palabra,0)+1  
  
for clave in frecuencia.keys():  
    print(frecuencia[clave],clave)
```

y por pantalla nos mostrará las palabras y las frecuencias, pero de una manera desordenada, puesto que los diccionarios de Python son estructuras de datos sin un orden determinado. NLTK proporciona una función `FreqDist` que facilita mucho el cálculo de frecuencias. Veamos su uso en el programa-4-16.py:

```
import nltk  
  
from nltk.corpus.reader.plaintext import PlaintextCorpusReader  
  
from nltk.tokenize import RegexpTokenizer  
  
from nltk import FreqDist  
  
segmentador= nltk.data.load("spanish-mod.pickle")  
  
tokenizador=RegexpTokenizer('\w+|[\^\w\s]+')  
  
corpus = PlaintextCorpusReader(".", 'DOGC-2015-  
spa.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)  
  
frecuencia=FreqDist(corpus.words())  
  
for mc in frecuencia.most_common():  
    print(mc)
```

Ahora sí que nos proporciona las palabras y la frecuencia de cada palabra ordenada de más frecuente a menos frecuente. Como que hay muchas, podemos modificar fácilmente el programa porque nos proporcione las 25 más frecuentes, cambiando la línea:

```
for mc in frecuencia.most_common():  
  
por  
  
for mc in frecuencia.most_common(25):
```

4.6.2. Frecuencia relativa

La frecuencia absoluta de una palabra en un determinado corpus no nos da información real sobre si la palabra es muy frecuente o no, porque esto dependerá del tamaño del corpus. Que una palabra aparezca, digamos, 22 veces en nuestro corpus, no nos dice nada, puesto que si el corpus es muy grande quizás este valor de frecuencia sea pequeño.

La frecuencia relativa de una palabra en un corpus es el número a veces que aparece dividida por el número total de palabras en el corpus.

`FreqDist` nos facilita mucho el cálculo de la frecuencia relativa puesto que la podemos consultar con el método **`freq()`**. Lo podemos ver en el programa-4-17.py, que es una modificación del anterior. Ahora guardamos al fichero

frecuencias.txt las palabras ordenadas por frecuencia y mostramos la frecuencia absoluta y la relativa de cada palabra:

```
import nltk

from nltk.corpus.reader.plaintext import PlaintextCorpusReader

from nltk.tokenize import RegexpTokenizer

from nltk import FreqDist

import codecs

segmentador= nltk.data.load("spanish-mod.pickle")

tokenizador=RegexpTokenizer('\w+|[\^\w\s]+')

corpus = PlaintextCorpusReader(".", 'DOGC-2015-
spa.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)

frecuencia=FreqDist(corpus.words())

salida=codecs.open("frecuencias.txt","w",encoding="utf-8")

for mc in frecuencia.most_common():

    palabra=mc[0]

    frecuencia_absoluta=mc[1]

    frecuencia_relativa=frecuencia.freq(palabra)

    cadena=str(frecuencia_absoluta)+"\t"+str(frecuencia_relativa)+"\t"+palabra

    salida.write(cadena+"\n")
```

Y la salida (mostramos solo las 25 primeras):

```
1779837 0.07246859684771821 de
1031818 0.042011938543933466 /
986574 0.040169764684317016 ,
921036 0.03750129172853188 .
668298 0.027210704315134695 la
421836 0.017175653174899757 y
394394 0.016058313084377378 :
386949 0.015755179312278437 el
377317 0.015362998722237202 -
349183 0.014217482866732621 en
302520 0.012317532402333312 del
290548 0.011830075381571926 2015
278339 0.011332968568468372 que
242310 0.009865996550341745 a
217715 0.008864576117195547 los
200890 0.00817952229374831 las
```

```
189023 0.007696340497442317 0
181949 0.007408312518419092 se
136654 0.005564062121210024 08
136191 0.005545210417182917 CIR
134874 0.005491586887585294 1
124829 0.005082590414686186 por
122338 0.004981165804034949 )
119835 0.004879252596303095 2
114850 0.004676281225730467 con
```

4.6.3. La ley de Zipf

La Ley de Zipf afirma que dado un corpus, la frecuencia de una palabra es inversamente proporcional a su posición a la tabla de frecuencias (rank). Con su ley, Zipf (1949) afirma que hay una constante k que se puede calcular multiplicando la frecuencia de cualquier palabra por su posición en la tabla (rank) ($k = f \cdot r$).

En el programa siguiente (programa-4-18.py) evaluamos la ley de Zipf con las 50 palabras más frecuentes del corpus Cess_esp:

```
import nltk
from nltk.corpus import cess_esp
import re
import codecs

palabras=cess_esp.words()
freedist=nltk.FreqDist(palabras)
salida=codecs.open("salida.txt", "w", encoding="utf-8")

posicion=0
p = re.compile('\w+')
for mc in freedist.most_common(50):
    if p.match(mc[0]):
        posicion+=1
        frec=mc[1]
        fxr=posicion*frec
        salida.write(mc[0]+"\\t"+str(frec)+"\\t"+str(posicion)+"\\t"+str(fxr)+"\\n")
```

En el fichero salida.txt podemos observar las palabras con su frecuencia, posición y resultado del producto de la frecuencia y la posición (que tiende a ser constante):

```
de 10234 1 10234
la 6412 2 12824
```


que 5552 3 16656
el 5199 4 20796
en 4340 5 21700
y 4235 6 25410
los 2963 7 20741
a 2953 8 23624
del 2257 9 20313
se 1884 10 18840
las 1832 11 20152
un 1815 12 21780
con 1494 13 19422
por 1456 14 20384
una 1396 15 20940
su 1291 16 20656
para 1258 17 21386
no 1232 18 22176
al 984 19 18696
es 911 20 18220
El 812 21 17052
ha 705 22 15510
como 696 23 16008
lo 652 24 15648
más 648 25 16200
La 510 26 13260
sus 493 27 13311
o 435 28 12180
pero 357 29 10353
hoy 348 30 10440
entre 315 31 9765
dos 306 32 9792
sobre 303 33 9999
En 301 34 10234
le 300 35 10500
años 291 36 10476
este 278 37 10286
han 271 38 10298

también 254 39 9906

fue 244 40 9760

si 227 41 9307

Los 226 42 9492

A pesar de que la ley solo muestra una tendencia y no es exacta, recalca el hecho que en un corpus hay muy pocas palabras muy frecuentes y muchas palabras poco frecuentes. En el programa siguiente (programa-4-19.py) graficamos este fenómeno con un subconjunto del corpus Cess_esp de 1.000 palabras. Cómo podemos observar en el gráfico, el principio mencionado se cumple, es decir, que muy pocas palabras aparecen muchas veces y que muchas aparecen pocas veces.

Para poder ejecutar este programas nos tenemos que asegurar de tener instaladas las librerías **scipy** y **matplotlib**. Recuerda que se pueden instalar con pip (o pip3).

```
import nltk
from nltk.corpus import cess_esp
import pylab
palabras=cess_esp.words()
palabras1=palabras[:1000]
freqdist=nltk.FreqDist(palabras1)
freqdist.plot()
```

Que crea el gráfico:

